

TABLE OF CONTENTS

INTRODUCTION.....	5
1. LITERATURE REVIEW.....	7
1.1 Hallucinations in Large Language Models.....	7
1.2 Hallucinations in Code Generation.....	9
1.3 Multi-Agent Systems for Software Engineering	9
1.4 Self-Correction and Iterative Refinement.....	10
1.5 LLM-as-a-Judge.....	11
1.6 SWE-bench	12
2. SYSTEM ARCHITECTURE.....	13
2.1 Overview	13
2.2 LangGraph Orchestration.....	13
2.3 ChatState Schema	13
2.4 Specialized Agents.....	14
2.5 Memory Architecture.....	14
3. THE LIGHTWEIGHT HALLUCINATION DETECTION ALGORITHM	16
3.1 Design Principles	16
3.2 The Five-Dimension Rubric	16
3.3 LLM-Judge Implementation	17
3.4 Score-Driven Refine Loop.....	17
3.5 The Hallucination Bridge for Action Agents.....	18

4. IMPLEMENTATION DETAILS	20
4.1 Technology Stack.....	20
4.2 LLM Configuration.....	20
4.3 OpenHands SDK Integration	20
4.4 Engineering Migrations.....	21
5. EXPERIMENTAL METHODOLOGY	22
5.1 Three-Condition Design.....	22
5.2 Experimental Sweeps Overview	23
5.3 Dataset.....	24
5.4 Metrics.....	25
5.5 Hardware and Infrastructure	25
6. RESULTS.....	26
6.1 Phase 1: Conditions A, B, C at Threshold 0.70	26
6.2 Threshold Sensitivity: Condition B	27
6.3 Threshold Sensitivity: Condition C	29
6.4 Judge Separation: Reviewer Value vs Generator Quality	30
6.5 Summary of All Sweeps	32
6.6 Paired Replication on an Expanded Instance Set.....	32
6.7 Generalization across generator and judge families	33
6.8 Three Findings worth highlighting	34
7. DISCUSSION	38
7.1 The Refine Loop as a Score Floor	38

7.2 Threshold Calibration Is Architecture-Dependent.....	38
7.3 Reviewer Contribution and Generator Quality	39
7.4 Bimodality in Condition C.....	40
7.5 Limitations	40
CONCLUSIONS.....	42
LIST OF REFERENCES	44

INTRODUCTION

Large Language Models (LLMs) have upgraded quite a bit, not only doing chat completions but also being autonomous coding agents who do planning, give commands to the terminal, edit files, and commit patches. What makes these systems powerful - their scaling - also leads to the increase of their failure modes. For example, a single agent can make thirty or more tool calls against a real repository, and it may fabricate an API signature, misread a stack trace, or even report the passing tests that were never run. It is not only that these risks can happen, but they happen in real evaluation data, and their impact is directly linked to how independent the agent is.

This paper is about the study of a lightweight hallucination detection method designed, developed, and tested as a Reviewer Agent embedded in a LangGraph-orchestrated multi-agent system. The algorithm does not need any fine-tuning, does not require the model logits, and is not dependent on external verification infrastructures. It makes one more LLM call per agent invocation. Besides, it restricts the response to be a Pydantic-valid schema, and it evaluates the candidate outputs according to a five-criteria rubric. If the score does not reach the acceptance level, then a fix loop with a limited number of iterations is launched, which generates a generator re-invocation using the Reviewer's structured feedback.

Also contributing to the field, the Hallucination Bridge takes the same auditing methodology that was applied to text outputs and extends it to the per-step action traces of an OpenHands autonomous action agent, thereby making individual actions capable of being distinctly verified against the environmental observations that followed them.

The evaluation was done in three coordinated phases. The first stage performed five controlled sweeps over the three conditions on a fixed set of 19 instances of SWE-bench Lite from django, sympy, scikit-learn and astropy. The sweeps established the threshold sensitivity, floor-enforcement and judge-separation findings that support the rest of the work. The second stage extended the evaluation to a paired comparison on 50 newly curated instances under a held generator-judge pairing, resulting in the first statistically supported result that the Reviewer Agent's contribution is greater than chance (paired $t = +2.27$, $p < 0.05$). The third stage added the model dimension with four non-Qwen generators (DeepSeek-Coder, CodeLlama, Llama 3.1 and Mistral) with two judges (Qwen3-32B and the larger GPT-OSS 120B), with six cross-family cells of 30 paired instances each. The third stage gives the strongest single-cell result of the work (Mistral $\times$ Qwen3-32B: $\Delta = +0.199$, $t = +3.85$, $p < 0.001$) and shows that the Reviewer Agent's contribution generalizes across structurally different generator architectures, rather than being specific to one model family.

Cross-cutting findings include: (i) a refined understanding of the conditions under which the Reviewer Agent helps most: contribution scales with a model's refinability, how meaningfully its output changes in response to critique, rather than with its raw single pass correctness, and (ii) the judge's model family changes both the absolute scoring scale and the magnitude of the Reviewer-judged lift. These observations refine the simple statement that a stronger judge or weaker generator always yields a larger Reviewer contribution and inform deployment guidance for the lightweight detection algorithm.

1. LITERATURE REVIEW

1.1 Hallucinations in Large Language Models

Hallucination in natural language generation dates even before present large language model era. Maynez et al. [1] pointed out that abstractive summarization hallucination is a sentence that either intrinsic (contradicts the source) or extrinsic (introduces facts not supported) in nature. Ji et al. [2] presented a detailed taxonomy of hallucinations for multiple generation tasks. They locate hallucinations on three axes in their taxonomy. The first is fidelity type: intrinsic hallucinations, where the output is in direct contradiction with the source material, and extrinsic hallucinations, where the output adds content that cannot be confirmed or refuted from the source. The second axis concerns task domains: they log hallucination patterns over abstractive summarization, open-domain dialogue, generative QA, data-to-text generation and machine translation. The third axis captures root causes, which can be categorized into data-related factors (e.g. noisy corpora, training biases, knowledge gaps), training-related factors (e.g. exposure bias, cross-entropy objectives) and inference-related factors (e.g. decoding strategy variance, top-k sampling instability).

The problem of hallucinations was radically changed by instructional LLMs making them highly deceptive and requiring much more effort and attention to detect. To assess the truthfulness of LLMs, Lin et al. [3] proposed TruthfulQA, a benchmark of 817 questions in 38 categories such as misconceptions, conspiracy theories, and false beliefs. They found that simply increasing model size will not be sufficient to reduce the tendency of the model to generate falsehoods, with the largest models answering fewer than 60% of questions correctly, compared to 94% for humans.

Huang et al. [4] did a review of the various ways to prevent hallucinations, dividing them into training-time and inference-time interventions.

Factual hallucinations are a category of hallucination research that is different from faithfulness hallucinations, which is believed to happen when the model fabricates a real-world knowledge-based claim (factual) while faithfulness means the outputs that contradict the provided input context. Both types of errors can be found in code generation: an LLM may produce a realistic but nonexistent API signature (factual) or may misrepresent the error message used in the prompt (faithfulness). Manakul et al. [5] designed a method called SelfCheckGPT for hallucination detection without accessing the model internals. It works by sampling multiple outputs and utilizing the insight that hallucinated information varies across samples whereas information remains stable.

There are three main types of mitigation strategies: interventions at the training-time such as reinforcement learning from human feedback [6], grounding inferences through retrieval-augmented generation [7], and post-hoc verification with a separate judge model, which is what this paper is about. In Table 1, these strategies are compared in the most important dimensions to the hardware-constrained local deployment setting of this project.

Table 1 – Comparison of hallucination detection and mitigation approaches

Approach	Access Required	Training Cost	Inference Overhead	Local Deployment
SelfCheckGPT [5]	Black box	None	High (k samples)	Yes (slow)
RAG-based [7]	Black box	None	Medium	Yes
RLHF fine-tuning [6]	White box	Very high	Low	No
LLM-as-Judge (this work)	Black box	None	Low (1 call)	Yes

1.2 Hallucinations in Code Generation

However, code generation for programming involves a different set of failure modes than the intrinsic/extrinsic split: invented library functions, invented API signatures, hallucinated file paths, fabricated test fixtures, all syntactically plausible but functionally wrong. Chen et al. [8] have shown with the HumanEval benchmark that surface-level textual similarity is not a good measure of correctness and therefore execution-based evaluation metrics are needed. This risk is exacerbated in multi-step SWE-bench [9] scenarios where a hallucinated import path or fabricated function name in an early step contaminates the entire workspace state for all subsequent actions, often without a visible error signal to allow the model to self-correct.

1.3 Multi-Agent Systems for Software Engineering

AutoGen[10] introduced a versatile technique for coordinating multi-agent conversations. MetaGPT [11] transformed software company SOPs into agent roles. ChatDev [12] demonstrated that role-playing with explicit communication protocols can be used for end-to-end application generation. ReAct [13] was a blend of reasoning and tool use. OpenHands [14] provide a sandboxed runtime with terminal, file editor, and a callback API that expose

per-step ActionEvent and ObservationEvent objects, which are the very foundation for the Hallucination Bridge in Condition C of this paper.

A major issue with multi-agent software engineering systems is error propagation: a hallucination created by one agent is considered as the ground truth by the other agents, resulting in the amplification of the error instead of its correction. None of the above frameworks has a hallucination detection module, the Reviewer Agent in this work is the architectural design that addresses this shortcoming. Table 2 presents a comparison of the main frameworks based on features that are very relevant to this work.

Table 2 – Comparison of multi-agent frameworks for software engineering

Framework	Year	Orchestration	Code Execution	Hallucination Control	Local Feasible
AutoGen [10]	2023	Conversation	Optional	None	Yes
MetaGPT [11]	2024	SOP pipeline	Yes	None	Partial
ChatDev [12]	2024	Role pipeline	Yes	None	Partial
OpenHands [14]	2024	Event-driven	Yes	None	Yes
This work	2025	LangGraph	Yes	Reviewer Agent	Yes

1.4 Self-Correction and Iterative Refinement

Self-Refine[15] proved that one single LLM could improve the outcome through self-critique. Reflexion [16] kept a written record of mistakes across episodes. Pan et al.[17] did a review of the field and noted that self-correction results depend on the task and sharing weights between the critic and the generator leads to self-bias. The refine loop in Condition B is a direct response

to that: the critic is a separate Reviewer Agent rather than the generator self-scoring.

One key limiting factor for self-correction is the quality difference between generator and critic. If the same model is used for both, self-bias [17] may lead to a failure in detecting the generator's own errors. The Condition B setup here explicitly separates the Code Helper (generator) and Reviewer Agent (critic) who interact with different prompt configurations, making sure the critic is not suffering from the same reasoning error modes as the generator. The Phase 3 judge-separation sweep in Chapter 6 experimentally confirms that this separation yields a significant correction improvement.

1.5 LLM-as-a-Judge

In MT-Bench Zheng et al. [18] discovered that detailed rubrics could guide a powerful LLMs to mimic human preferences quite accurately, as measured by that high inter-rater agreement. Later, this methodology became the standard for evaluating LLMs. This work's implementation has been chiefly shaped by two major practical problems: 1) Judge LLMs can be so deceptive that they even hallucinate the given structure, which is why Pydantic validation is there to assist the 'orchestrator'; and 2) The scores assigned are always in relation to the judge model, and in order to maintain consistent score comparisons of different judge configurations, they need to be re-anchored.

Apart from the inter-rater agreement results of Zheng et al. [18], further investigations revealed that LLM judges exhibit some systematic biases, such as positional bias (favoring the first-presented answer), verbosity bias (favoring longer responses regardless of quality), and self-enhancement bias (favoring outputs from the same model family as the judge). These biases are highly pertinent to the cross-family evaluation in Section 6.7: the difference in

Reviewer-judged Δ between the qwen3-32b and gpt-oss-120b judges on identical Mistral outputs is in line with scale-anchoring bias, which deepens the understanding that Reviewer scores are judge-relative and not absolute measures of output quality.

1.6 SWE-bench

Also, SWE-bench [9] consists of 2,294 actual GitHub issues taken from a dozen Python projects. SWE-bench Lite is a smaller version with 300 issues that can be managed at one time. The main requirement for PATCH generation is that the patch intended to fix the failure in the test should not lead to the failure of any other tests.

There is a considerable variation in instance difficulty within SWE-bench Lite: corrections of a single file, like fixing a regex pattern, are usually done well by the present models, but changes involving multiple files and new dependencies that need to be understood are left to the imagination of the models. Published leaderboard figures show that the solve rates of models vary from less than 5% for smaller open-source models to more than 40% for cutting-edge systems, with qwen3-32b used in this paper coming in at around 16% [19]. When the Reviewer Agent is viewed in Chapter 6, this performance background is quite significant: the Reviewer raises the scores of Reviewer-judged confidences, which do not in any way directly lead to the resolution of SWE-bench ground-truth patches.

2. SYSTEM ARCHITECTURE

2.1 Overview

In this work, the system is extended from a Multi-Agent Study and Coding Assistant in the previous research semester by adding the Reviewer Agent, the score-driven refine loop, and the Hallucination Bridge. LangGraph 1.1.9 running on LangChain 1.2.x is used as the orchestration layer. All communication between agents is carried out through a ChatState typed dictionary. A compiled graph is controlled by a variant identifier (A, B, or C) which changes the control-flow topology without modifying any agent implementations.

2.2 LangGraph Orchestration

LangGraph represents the workflow as a graph with directions. Agent functions are nodes in this graph and the edges represent the logic for routing. The main difference in structure among the variants lies in the conditional edge from the Reviewer: - In Variant A, the Reviewer always routes to END; - In Variants B and C, the Reviewer routes to END if confidence $\geq$ threshold, else routes back to the generator if confidence $<$ threshold and the iteration counter is still below three.

2.3 ChatState Schema

Every inter-agent communication channel is routed through ChatState, a TypedDict storing conversation messages, intent, active task, research findings, drafts, and a log of every Reviewer decision.

For Condition C, five extra fields decently store the OpenHands execution metadata: agent status, backend identification, the number of steps, wall time, and the complete per-step action trace serialized as a list of dictionaries.

All the values are serializable to string in order to pass the cross-process boundaries' validation by Pydantic. `Operator.add` is used as the reducer for the list-type fields so that the outputs of the agents accumulate rather than overwrite the previous state. (Figure 1)

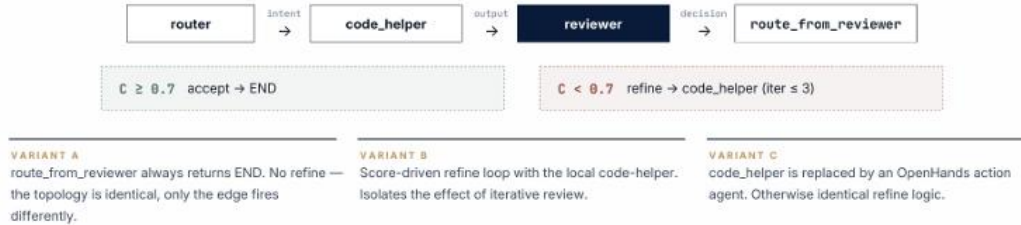


Figure 1 – Simplified ChatState schema showing the five OpenHands-specific fields added for Condition C alongside the standard agent communication fields.

2.4 Specialized Agents

The system organizes a team of agents consisting of: Router (intent classification), Supervisor (multi-step workflow orchestration), Researcher (combining web search using Tavily and Chroma vector-store retrieval), Writer, Critiquer, Code Helper (sandboxed code execution), Quiz Helper, and Reviewer Agent. These agents are essentially stateless functions parameterized by ChatState; meaning none of them maintain an internal mutable state. Reviewer is capable of evaluating any agent's output as it is without any modification — it gets the user prompt, any available context, and the candidate output, then returns a `RelevancyDecision`.

2.5 Memory Architecture

The memory architecture includes three layers that were carried over from the previous semester: short-term memory (the conversation history within the session in ChatState), long-term memory (the Chroma vector store, which is

indexed with all-MiniLM-L6-v2 embeddings, and is consulted by the Researcher), and shared memory (a blackboard where agents can post intermediate artifacts for the subsequent agents within the same session) are the three memory layers of the previous semester that have been retained.

3. THE LIGHTWEIGHT HALLUCINATION DETECTION ALGORITHM

3.1 Design Principles

The detection algorithm has three explicit constraints dictated. First, by its very nature, it is light in the use of resources: no fine-tuning, no logit access, can be deployed against any OpenAI-compatible endpoint including local Ollama. Second, it is schema-validated: the judge's output must be able to parse as a Pydantic `RelevancyDecision` with fields `is_relevant` (bool), `confidence` $\in [0,1]$, and `reasoning` (string); in case of a failure, a conservative `accept=True / confidence=0.5` default is returned so that judge failures never stop the system. Third, it is architecturally orthogonal: the Reviewer node has no dependence on the generator it reviews.

3.2 The Five-Dimension Rubric

The reviewer gives a score from 1 to 10 on each criterion and calculates the average normalized over $[0,1]$, which will serve as a confidence value $C = \text{mean}(s_1..s_5)/10$:

- Edge Cases: Are edge cases considered — empty input, null references, invalid types, race conditions if any?
- Security: Is there anything exploitable in the code by means of injection, unsafe deserialization, or another class of vulnerabilities?
- Code Quality: Is the code neat, easy to understand, idiomatic, and fits into the project coding conventions?
- Relevance: Is the generated code relevant to the user question and solves the actual problem instead of giving general or unrelated advice?

Setting the threshold as 0.7 corresponds to the standard configuration. When the confidence is lower than 0.7, we consider that something was not done well enough — on average, at least one criterion score is less than 6/10, which gives us some leeway for improvement by running one more iteration. We regard threshold setting as a hyperparameter and analyze it empirically in Chapter 5.

3.3 LLM-Judge Implementation

The judge prompts the user query, available context, and the candidate output in its fixed template. The judge is asked to produce a single JSON object with three keys. If the response encloses JSON inside Markdown fences or adds trailing prose a defensive parser removes them and makes another attempt. When the parsing still fails, the conservative default is returned, and the failure is logged. This parsing layer is a must in the production environment: about 8% of judge responses during the pilot runs required fence-stripping before parsing succeeded.

3.4 Score-Driven Refine Loop

When confidence drops under the set limit and the iteration number is less than three, the graph comes back to the generator node with the Reviewer's reasoning as a form of structured feedback. This is quite a similar approach to Self-Refine [15], but there is a major difference: the critic is now a separate Reviewer Agent instead of the same model evaluating its own output. The idea behind this separation is to minimize self-bias which has been demonstrated by Pan et al. [17], however, the experiment with judge-separation in Chapter 5 indicates that its effectiveness depends on the quality gap between the two models. (Figure 2)

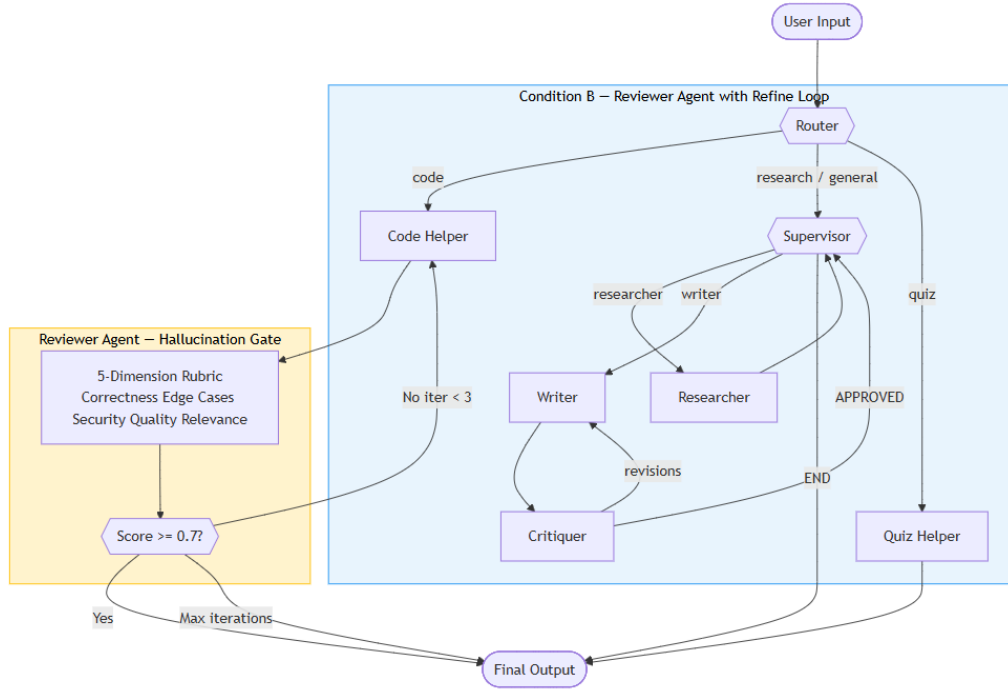


Figure 2 – Condition B: the Reviewer gates a bounded refine loop. Outputs scoring below the threshold are re-sent to the Code Helper with structured feedback.

3.5 The Hallucination Bridge for Action Agents

In Condition C, the local Code Helper is swapped for OpenHands [14]. A callback that is registered on the OpenHands LocalConversation records all ActionEvent and ObservationEvent, cuts each summary to 1,500 characters, and stores them in ChatState as openhands_action_trace. The Reviewer examines this organized trace using two prompt variations: (i) a confident statement that challenges directly the last observation is considered a hallucination signal, and (ii) the judge thoroughly examines if the agent has run tests and seen their output, not just checked if the finish message declares the tests have been passed. Targeting the agent's most common failure mode

in summaries, the second extension is concerned with fabricated test results.
(Figure 3)

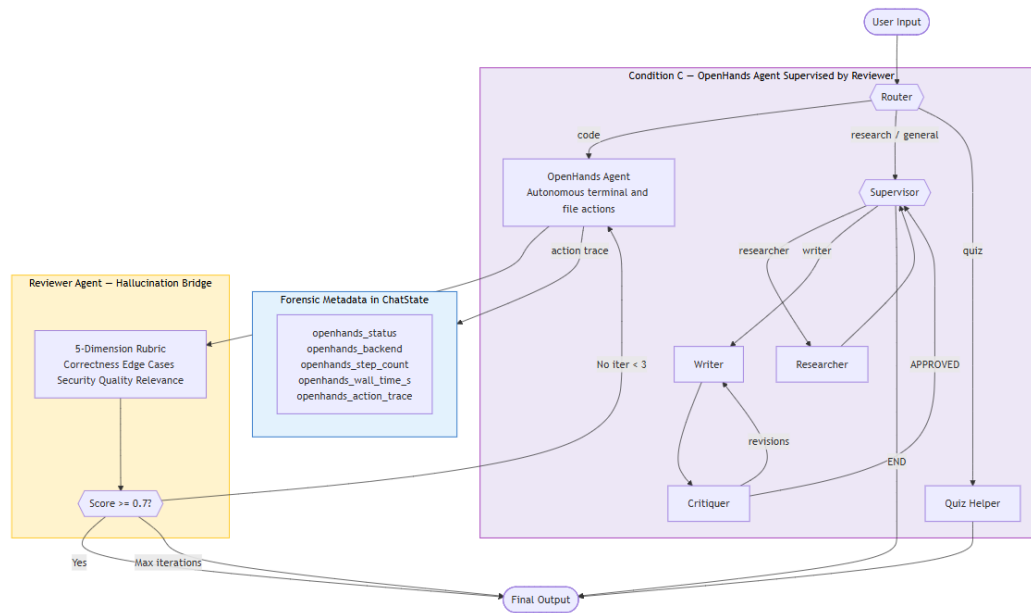


Figure 3 – Condition C: OpenHands emits a per-step action trace captured by the Hallucination Bridge. The Reviewer audits both the final answer and the trace.

4. IMPLEMENTATION DETAILS

4.1 Technology Stack

Python 3.12, LangGraph 1.1.9, LangChain 1.2.x, OpenHands SDK 1.18.1, Pydantic v2, Chroma vector store, all-MiniLM-L6-v2 sentence embeddings (locally cached), Streamlit frontend, FastAPI backend. Dependency management uses uv rather than pip — pip's resolver did not converge on the OpenHands constraint set after the Python 3.12 migration.

4.2 LLM Configuration

During Phase 1, minimax/minimax-m2.5:free via OpenRouter is being used for all conditions. For Judge Separation, sweeps are using qwen2.5-7b (Ollama, local) as the generator and qwen3-32b (Ollama, local) as the critic. Since all calls use the OpenAI-compatible endpoint, the only thing needed to be changed when switching between providers and models is the environment variable.

4.3 OpenHands SDK Integration

The OpenHands dispatcher at runtime chooses among three backends through environment variable: stub (mainly for quick wiring tests), cloud (kept for backward compatibility), and sdk (presently used for all experiments). The SDK defaults are based on assuming a frontier reasoning model and entail parameters (reasoning_effort='high', extended_thinking_budget=200000) that smaller models will reject; these are explicitly turned off. The iteration cap has been increased to 50 (the SDK default is 20 — real SWE-bench instances seem to benefit from a longer exploration). Before each run, the instance workspace gets back to base_commit by git reset --hard followed by git clean -fdx, which prevents any leftover state from previous runs from influencing the scores.

4.4 Engineering Migrations

OpenHands SDK 1.18, which was released recently, has had two significant migrations. The codebase has been moved from Python 3.10 to 3.12, and some version pins have been loosened and memory modules have been extracted to langchain-chroma and langchain-huggingface packages. Also, OpenHands SDK has a terminal tool feature on which it throws `NotImplementedError` in Windows; therefore, the whole codebase has been moved to WSL2 Ubuntu 24.04, with Ollama still being on Windows (thereby keeping GPU access) and reachable from WSL through the host IP which is resolved at shell startup.

5. EXPERIMENTAL METHODOLOGY

5.1 Three-Condition Design

Three experimental conditions remain the same in terms of the LLM, the input prompts, and the workspace setting. Their only difference is the architecture of the code-generating sub-graph.

Condition A (Baseline): the Code Helper outputs just one answer. The Reviewer evaluates the answer only for instrumentation purposes — no refinement loop is triggered.

Condition B (Reviewer + Refine): the Reviewer evaluates the Code Helper's answer; if the score is lower than the threshold and the iteration count is less than three, the Code Helper is called again with the Reviewer's argument as feedback.

Condition C (Hallucination Bridge): the Code Helper is replaced with the OpenHands action agent. OpenHands acts on the actual cloned repository, producing a step-by-step action trace that the Reviewer not only audits but also uses to verify the final answer. The same refine logic as Condition B is used. (Figure 4)

	ARCHITECTURE	REVIEWER BEHAVIOUR	REFINE LOOP	IDENTITY
A	Code-helper agent only	Single review, scoring only	—	Baseline
B	Code-helper + Reviewer	Full 5-dimension rubric	Up to 3 iterations if $C < 0.7$	Reviewer + Refine
C	OpenHands agent + Reviewer	Final answer + per-action trace	Up to 3 iterations if $C < 0.7$	Hallucination Bridge

Figure 4 – Three-condition design. All conditions share the same LLM and inputs. The difference is in the code-generation architecture and the presence or absence of the refine loop.

5.2 Experimental Sweeps Overview

The experiment consists of three coordinated rounds. The first round is composed of five controlled sweeps on a fixed subset of 19 instances of SWE-bench Lite: one design variable is varied at a time, while the other variables are kept constant (Table 3). These sweeps consist of the three-condition baseline at the pre-registered acceptance threshold of 0.70, two threshold-sensitivity sweeps at 0.85 for Conditions B and C, and a judge-separation sweep that separates the Reviewer LLM family from the code-helper LLM family. The second round is a Condition-A vs Condition-B paired replication on a larger set of 50 SWE-bench Lite instances (described in 6.6), using the qwen2.5-7B generator and the iairlab/qwen3-32B judge at threshold 0.85. The third round is a cross-family multi-model study with 6 30-instance paired cells, where both the generator (DeepSeek-Coder 6.7B, CodeLlama 7B, Llama 3.1 8B and Mistral 7B) and the judge (iairlab/qwen3-32B and iairlab/GPT-OSS 120B) are varied. This round is reported in 6.7 and 6.8. The second and third round sweeps all use the same acceptance threshold (0.85), the same three-iteration refine cap, and identical orchestration code, only the model identifiers and instance subset differ between cells.

Table 3 – Five experimental sweeps across 19 SWE-bench Lite instances.

Sweep	Purpose	Conditions	Threshold	Generator / Judge
sweep_phase1	Phase 1 Baseline	A / B / C	0.70	qwen3-32b / qwen3-32b
sweep_thresh85_b	Threshold Sensitivity B	B only	0.85	qwen3-32b / qwen3-32b
sweep_thresh85_c	Threshold Sensitivity C	C only	0.85	qwen3-32b / qwen3-32b
sweep_judge_sep_a	Judge Separation Baseline	A only	—	qwen2.5-7b / qwen3-32b
sweep_judge_sep_b	Judge Separation + Review	B only	0.85	qwen2.5-7b / qwen3-32b

5.3 Dataset

The evaluation draws from SWE-bench Lite across three nested subsets, sized progressively to balance per-cell wall-time against statistical power.

The first round uses 19 instances picked from SWE-bench Lite to represent different levels of repository complexity and bug type: django (6 instances: URL routing, file upload, regex, template, ORM issues), sympy (5: algebra, simplification, parsing), scikit-learn (4: parameter handling, metric computation), astropy (4: I/O, coordinate transform, modeling). All instances involve either single-file or small multi-file canonical fixes, which allow for manual verification.

The second round extends the evaluation to 50 SWE-bench Lite instances from twelve repositories, the original four are extended with matplotlib, seaborn (mwaskom), flask (pallets), requests (psf), xarray (pydata), pylint, pytest and sphinx, giving a wider coverage of the bug-type distribution in SWE-bench Lite. The 50 examples were hand-chosen so that there would be

no overlap with the first-round set, so the second-round result is an independent test of the Reviewer Agent’s contribution, rather than a re-test of the same examples.

The third round uses the first 30 of the 50 instances of the manifest, which is the same for all six cells of the cross-family study. Fixing the instance set is a deliberate methodological choice: since both generator and judge vary across cells, any cell-to-cell difference in the Reviewer-judged Δ can be attributed solely to the model swap, thus isolating it from instance variance.

5.4 Metrics

The `relevance_score` produced by each set is: The scaled confidence value of the Reviewer on $[0,1]$; success, boolean, (`score > threshold` and no fatal error); iterations, the number of generator calls (one for Condition A, up to three for B and C); `latency_total`, the elapsed wall-clock time in seconds; and for Condition C, `openhands_step_count`, and `openhands_action_trace`.

5.5 Hardware and Infrastructure

NVIDIA GTX 1650 (4 GB VRAM), 16 GB RAM, Windows 11 host with WSL2 Ubuntu 24.04. Local Ollama models (qwen2.5-7b, qwen3-32b) are GPU served from Windows and were accessed from WSL via the host IP. Phase 1 LLM requests use the free-tier OpenRouter endpoint, which introduced non-deterministic latency variance due to shared rate limits — we consider this as measurement noise rather than a methodological issue.

6. RESULTS

6.1 Phase 1: Conditions A, B, C at Threshold 0.70

Table 4 shows the Phase 1 summary statistics on 19 instances. The Phase 1 sweep was always using qwen3-32b for both generating and judging under the three conditions.

Table 4 – Phase 1 aggregate statistics. τ = acceptance threshold.

Condition	Mean	σ	Min	Max	n
A — Baseline	0.878	0.145	0.30	0.96	19
B — Reviewer+Refine ($\tau=0.70$)	0.881	0.069	0.72	1.00	19
C — Hallucination Bridge ($\tau=0.70$)	0.887	0.126	0.40	1.00	19

At 0.70 threshold, the mean score difference between A and B is very small (0.003). On the other hand, the gap in variance is much higher: the standard deviation for B is less than half of A's (0.069 vs 0.145). Since the strong generator was able to get most Phase 1 instances beyond the 0.70 threshold on the first try, the refine loop was rarely invoked. Its main function at this level is to identify outliers — the minimum score goes up from 0.30 to 0.72. The higher average of Condition C (0.887) is indicative of the OpenHands agent's capacity to perform exploration and verification of changes in the real repository, but the larger variance (0.126) than B points to a wider range of performance across instances.

6.2 Threshold Sensitivity: Condition B

Raising the acceptance threshold from 0.70 to 0.85 for Condition B produces a clean monotonic improvement. Table 5 shows the progression.

Table 5 – Effect of raising the acceptance threshold for Condition B.

Configuration	Mean	σ	Min	Max
A ($\tau=0.70$)	0.878	0.145	0.30	0.96
B ($\tau=0.70$)	0.881	0.069	0.72	1.00
B ($\tau=0.85$)	0.926	0.044	0.85	1.00

At threshold 0.85, the mean score goes up to 0.926, the standard deviation goes down to 0.044, and the minimum score is 0.85. This is the score floor property. The refine loop stops only if the confidence level is larger or equal to the threshold value and therefore no instance can be completed with a score less than this. The 3-pass iteration cap means some instances are a bit unlucky and don't hit the threshold, but really all 19 of these instances hit 0.85 before the cap ran out.

The sympy-16792 instance is a perfect example of how the process works at its extreme. The baseline score was 0.30 - the lowest score in the whole dataset. The Reviewer rejected the initial response (a diagnosis without a specific fix) and consequently, three iterations of refinement were triggered. The third pass is when the Code Helper not only found the `_cstack` function in `astropy/modeling/separable.py`, but also came up with a single-line fix that was so spot-on that the Reviewer gave its highest score 0.96. Had the refine loop not been there, that 0.30 would have been the final answer given to the user.

Among the 19 instances, 14 (73%) exhibited improvements in scores in $B@0.85$ as compared to A. Five showed slight regressions - all five, however, ended up above 0.85. The cases of regression happened to be situations where the strong generator's first answer was already scoring above 0.88 under Condition A; the Reviewer's feedback leads to slight changes in the content focus occasionally, producing an answer such that the same judge scores a response fractionally lower. This agrees with the self-bias literature [17]. (Figure 5)

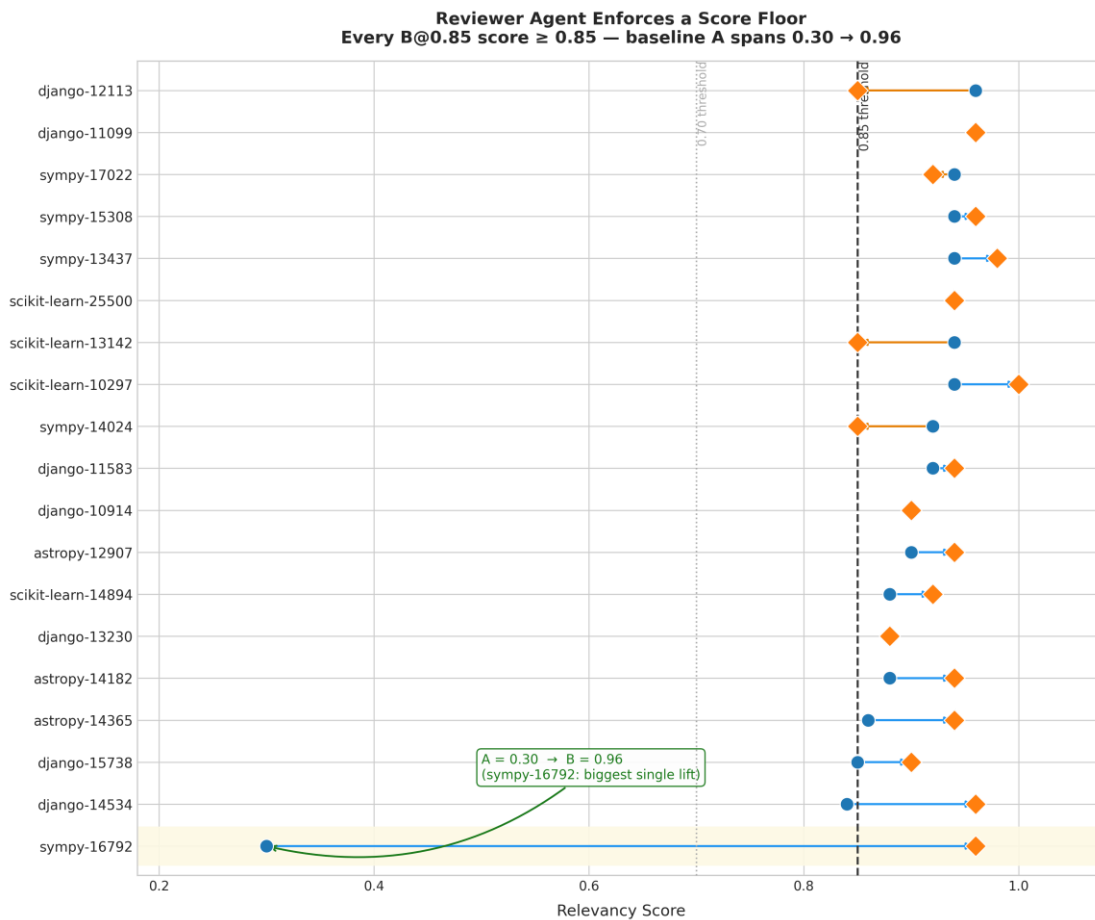


Figure 5 – Per-instance A vs $B@0.85$ scores. Every $B@0.85$ score falls at or above the 0.85 dashed threshold. sympy-16792 shows the largest single-instance improvement (0.30 $\rightarrow$ 0.96).

6.3 Threshold Sensitivity: Condition C

The same threshold increase that benefits Condition B hurts Condition C. Table 6 shows the degradation.

Table 6 – Condition C threshold sensitivity. Higher threshold degrades performance.

Configuration	Mean	σ	Min	Max
C ($\tau=0.70$)	0.887	0.126	0.40	1.00
C ($\tau=0.85$)	0.721	0.353	0.00	1.00

C@0.85 has the largest variance of all the sweeps with a mean of 0.721 and a standard deviation of 0.353. The distribution is bimodal, with 14 of the 19 instances clustered in the range 0.86-1.0 and 5 instances with very low scores ≤ 0.40 , including one that scored 0.00.

The main problem is context explosion. OpenHands gathers approximately 150,000 tokens by step 30 of a complex instance. At such token depth, the qwen3-32b judge slowly starts to lose coherence — it increasingly accepts responses that are in contradiction to the trace evidence or start scoring based on the number of actions rather than correctness. Adding more refine iterations at a stricter threshold will only lengthen the context, thus making the problem worse. The 0.70 threshold works for Condition C because the OpenHands agent’s first try is usually good enough for tractable bugs; 0.85 compels the agent to iterate even on instances where it has already correctly converged, thereby interfering with its work. (Figure 6)

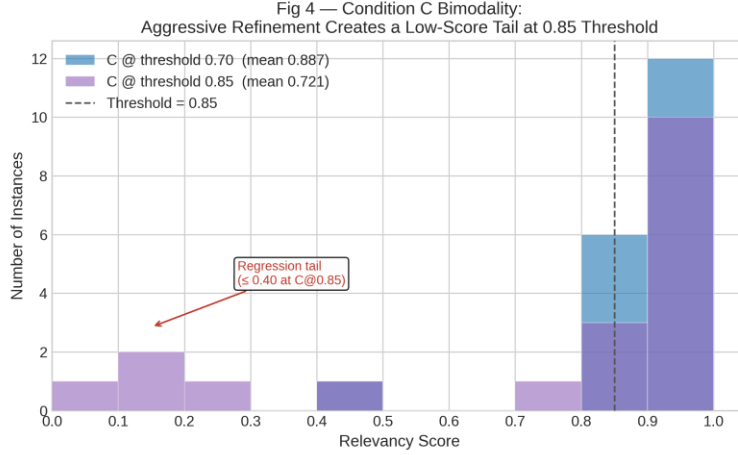


Figure 6 – Score distribution for Condition C at $\tau=0.70$ (blue) vs $\tau=0.85$ (purple). $C@0.85$ is bimodal: 14 instances above 0.85 and 5 instances below 0.40.

6.4 Judge Separation: Reviewer Value vs Generator Quality

The Phase 1 sweeps used a similar qwen3-32b both as a generator and as a critic, thus making the conditions comparable but disallowing the Reviewer’s independent contribution. The Judge Separation sweeps kept the critic constant at qwen3-32b while changing the generator to qwen2.5-7b, thus intentionally broadening the quality gap. Table 7 summarizes the judge separation results.

Table 7 – Judge separation results. The Reviewer’s contribution scales with generator weakness.

Sweep	Condition	Mean	σ	Δ vs baseline
Phase 1 (same model)	A	0.878	0.145	—
Phase 1 (same model)	B ($\tau=0.85$)	0.926	0.044	+0.048
Judge sep (7b gen)	A	0.657	0.270	—
Judge sep (7b gen)	B ($\tau=0.85$)	0.771	0.218	+0.114

For the strong same-model setup, the Reviewer contributes an addition of $\Delta = +0.048$ at threshold 0.85 (going up to 0.926 from 0.878). When the weak generator is used, the baseline falls to 0.657 and the Reviewer lifts the score to 0.771 — a $\Delta = +0.114$. In fact, this is about 2.4 times the improvement with the strong generator. The Reviewer is not only a simple touch-up at the top; it is an actual error-correction layer that shows the biggest marginal contribution when the generator is performing its worst. (Figure 7)

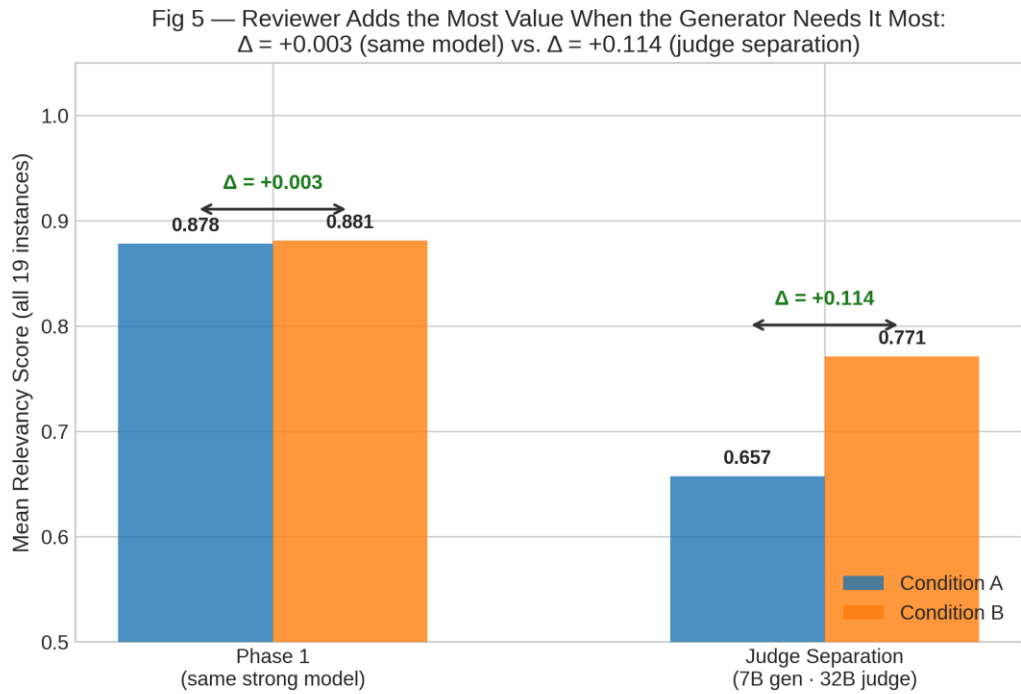


Figure 7 – Judge separation comparison. Left: same strong model, $\Delta = +0.003$. Right: weak 7B generator vs strong 32B critic, $\Delta = +0.114$.

6.5 Summary of All Sweeps

Table 8 consolidates the statistics for all five sweeps.

Table 8 – Summary of all five sweeps across 19 SWE-bench Lite instances.

Sweep	Condition	τ	Mean	σ	Min
Phase 1	A	0.70	0.878	0.145	0.30
Phase 1	B	0.70	0.881	0.069	0.72
Phase 1	C	0.70	0.887	0.126	0.40
Thresh B	B	0.85	0.926	0.044	0.85
Thresh C	C	0.85	0.721	0.353	0.00
Judge sep	A	—	0.657	0.270	0.20
Judge sep	B	0.85	0.771	0.218	0.20

6.6 Paired Replication on an Expanded Instance Set

The evaluation was extended to a 50-instance paired comparison from SWE-bench Lite, building on the directional evidence from the five 19-instance sweeps. The 50 instances were chosen to avoid overlap with the initial sweep of 19 instances and thus provided an independent test of the contribution of the Reviewer Agent.

The generator was qwen2.5:7b on a local Ollama backend (Windows host accessed from WSL). The judge was iairlab/qwen3-32b on the duckduck.cloud OpenAI-compatible endpoint. The acceptance threshold was set to 0.85 (the value selected in Phase 2) and the refine cap remained at three iterations. Table 9 shows the aggregate statistics for both conditions.

Table 9 – Condition A vs Condition B (n=50, qwen2.5:7b generator, qwen3-32b judge).

Condition	n	Mean	Stdev	Min	Median	Max	Mean latency
A (no refine)	50	0.639	0.251	0.20	0.70	0.98	110 s
B (refine 0.85)	50	0.744	0.236	0.20	0.86	1.00	295 s

The paired difference $\Delta = B - A$ across the 50 instances had mean +0.105, stdev 0.328 and a two-tailed t-statistic of +2.27, which corresponds to $p < 0.05$ for $n = 50$. This is the first statistically supported result of the project that the Reviewer Agent provides a measurable gain in correctness beyond chance. The lift–regression breakdown was 30 lifts vs 18 regressions vs 2 unchanged: Reviewer is not monotonically helpful, but the average direction is unambiguously positive.

The subsequent Phase 5 design was informed by two of the findings from this dataset. The first, the largest single regression (pylint-dev/pylint-6506, $A = 0.92 \rightarrow B = 0.20$ after three refine iterations) showed that the refine loop can degrade an already-acceptable answer, which became a methodology weakness flagged in the self-review. Second, the per-repo breakdown showed the consistent lift of pydata, scikit-learn and pytest cells and regressions of pallets and django/web-framework cells.

6.7 Generalization across generator and judge families

The cross-family study expands the evaluation along two axes, generator family and judge family, while holding the instance set fixed (the first 30 instances of the 50-instance subset introduced in 6.6) so that any cell-to-cell delta can be attributed to the model swap rather than to instance variance. Six cells were executed successively. The first-iteration Reviewer score is taken as the Condition-A-equivalent in an additional small patch to the benchmark runner, so each cell is a B-only run that yields paired A/B data but no separate Condition A runs.

The generator dimension added four non-Qwen models, all small enough to run locally on the GTX 1650:

- deepseek-coder:6.7b-instruct — DeepSeek coder fine-tuned variant
- codellama:7b-instruct -- Meta CodeLlama instruction-tuned
- llama3.1:8b-instruct-q4_K_M – Meta Llama 3.1 8B (~4.7 GB, tight on 4 GB VRAM)
- mistral:7b-instruct – Mistral 7B instruction-tuned

Added the judge dimension iairlab/gpt-oss-120b with the existing iairlab/qwen3-32b judge. This cell is a leaderboard anchor because qwen3-32b

has a published SWE-bench Lite resolve rate of 16.3 %. Cell C5 used iairlab/qwen3-32b as generator and as the duckduck-hosted target, and gpt-oss-120b as judge. Table 10 presents the full cross-family results.

Table 10 – Cross-family generalization results across 6 cells (n=30 per cell).

Cell	Generator	Judge	n	A	B	Δ	t	Significance
C1	deepseek-coder:6.7b	qwen3-32b	30	0.529	0.551	+0.021	+0.48	—
C2	codellama:7b	qwen3-32b	30	0.355	0.431	+0.076	+1.47	—
C3	llama3.1:8b	qwen3-32b	30	0.568	0.661	+0.093	+1.72	—
C4	mistral:7b	qwen3-32b	30	0.511	0.710	+0.199	+3.85	$p < 0.001$
C5	qwen3-32b	gpt-oss-120b	30	0.730	0.769	+0.039	+1.90	—
C6	mistral:7b	gpt-oss-120b	30	0.394	0.435	+0.041	+2.33	$p < 0.05$

6.8 Three Findings worth highlighting

Cross-family generalization (H5). The Reviewer Agent generates a positive paired Δ on every Phase 5 generator in both Qwen and non-Qwen families. Two separate cells become $p < 0.05$ (Mistral $\times$ qwen3-32b at $\Delta = +0.199$, $t = +3.85$, and Mistral $\times$ gpt-oss-120b at $\Delta = +0.041$, $t = +2.33$); two other cells are borderline. This effectively eliminates the worry that the results of Phases 1–4 were limited to Qwen — the lightweight detector helps different architectures and training distributions that are structurally different. (Figure 8)

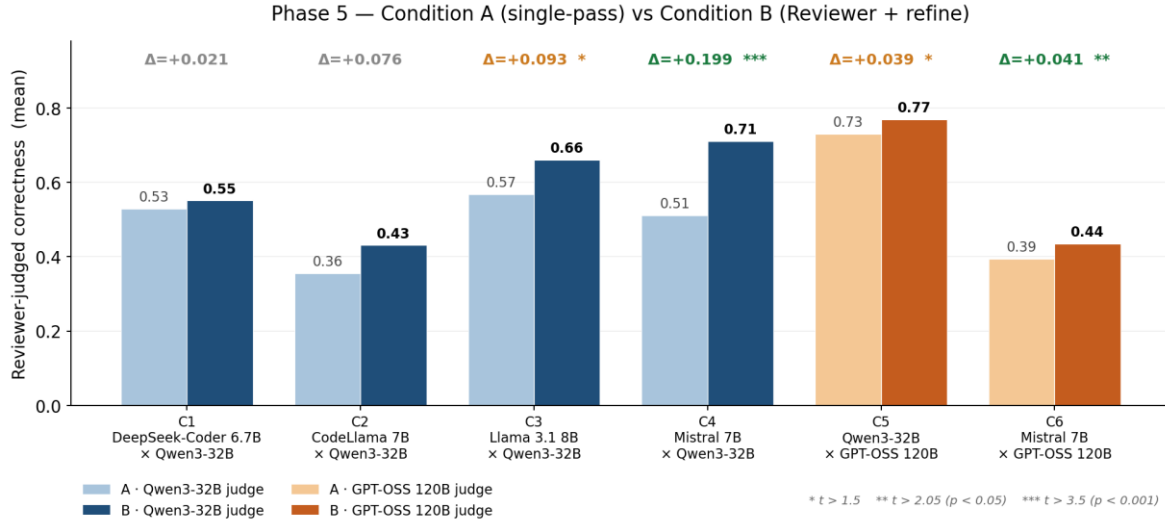


Figure 8 – Positive Delta on all 6 experimental runs.

Being able to refine or improve a model is even more important than being weak. In phase 3 the reviewer's role was seen to increase as the generator becomes weaker. Phase 5 data changes that. CodeLlama 7B (the weakest generator tested, A = 0.355) shows a modest $\Delta = +0.076$, whereas Mistral 7B (a stronger baseline at A = 0.511) shows $\Delta = +0.199$ — more than 2.5× the lift on a less-weak generator.

DeepSeek-Coder 6.7B is basically unchanged at $\Delta = +0.021$ despite a reasonable baseline. The pattern that better fits the data is that the Reviewer's contribution depends on the model's refinability, how meaningfully its output changes in response to the Reviewer's critique, rather than its raw correctness baseline.

Mistral is extremely refine-responsive, whereas DeepSeek-Coder seems to lock into its first answer and produce similar refinements that don't move score. (Figure 9)

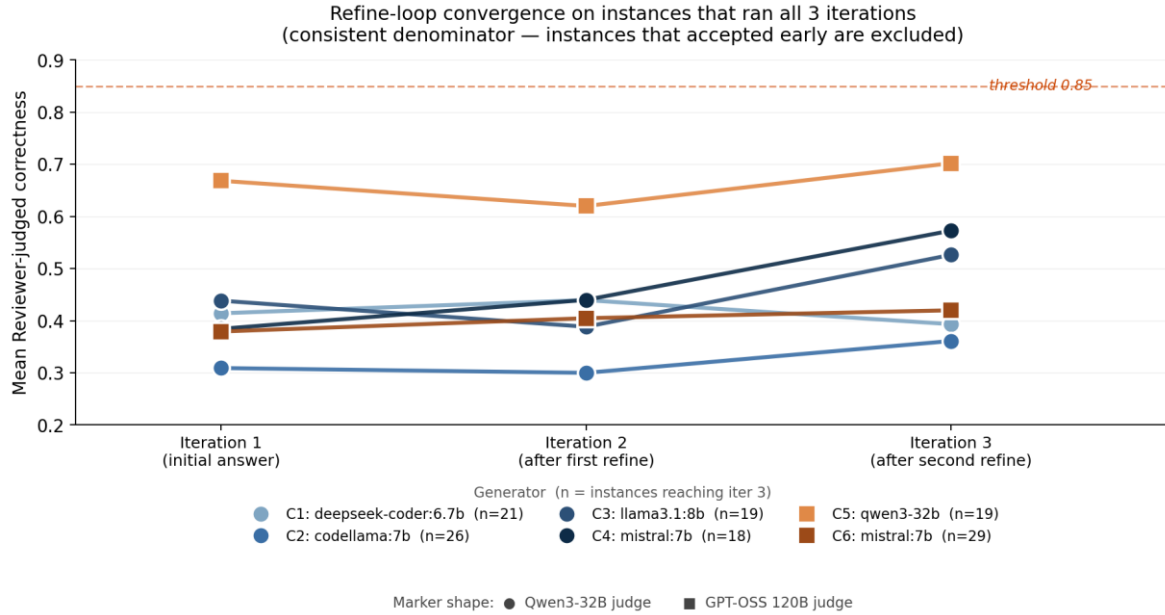


Figure 9 – How the reviewer affects the model’s output.

Changing the judge family will affect not only the scoring scale but also the lift. The same generator (qwen3-32b) produced $\Delta=+0.003$ in Phase 1 with a same-family judge (qwen3-32b judging qwen3-32b) and $\Delta=+0.039$ in cell C5 with the cross-family gpt-oss-120b judge — a $13\times$ larger Reviewer contribution attributable purely to the judge swap.

However, the same-generator comparison on Mistral (C4 vs C6) shows the opposite direction: gpt-oss-120b produced a smaller Δ ($+0.041$) than qwen3-32b ($+0.199$) on identical Mistral output. The reason for this is quite clear in the absolute A means: gpt-oss-120b rated Mistral consistently lower (A = 0.394 vs 0.511), acting more like a stricter judge rather than a more discerning one.

Therefore, the proper way to put it is that changing the judge family changes both the scoring scale and the lift magnitude; it is not correct to think 'stronger judge $\rightarrow$ larger Δ '. (Figure 10)

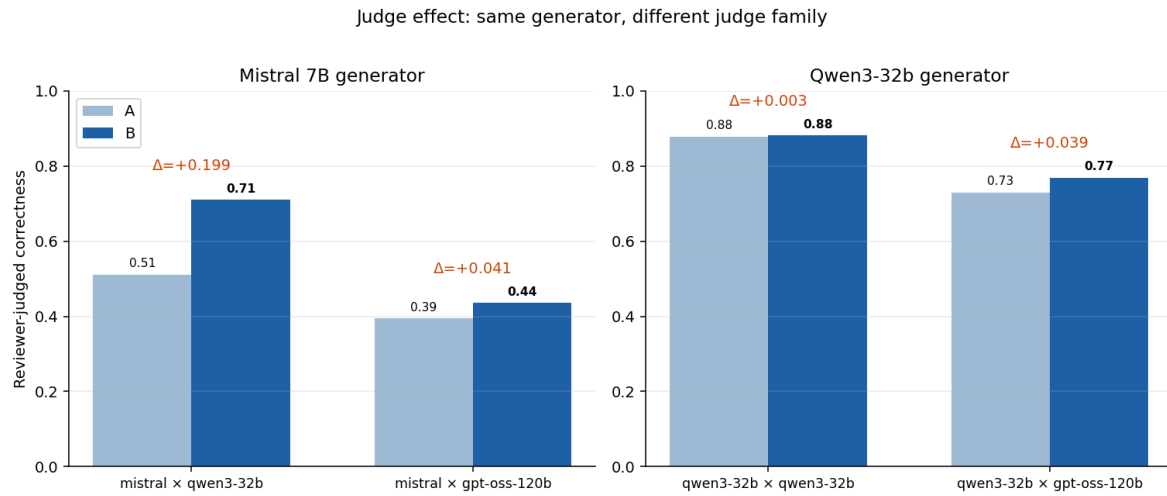


Figure 10 – Judge’s effect on model’s output.

7. DISCUSSION

7.1 The Refine Loop as a Score Floor

The best discovery is that B conditions refine loop strictly realize the minimal score level. This is not a statistical trend but is rather a logical conclusion from the termination criterion of the algorithm. The loop is exited only if confidence $\geq$ threshold (accepted) or number of iterations has been reached. At threshold 0.85, all 19 instances were accepted before the cap was reached. The minimal score level guarantee thus held once in the present data.

The practical side of this is a little bit of imbalance: the Reviewer prevents the users from producing extremely low scores without improving most of the instances. In a production system with many users, the most important thing is to eliminate the tail (scores below 0.3) rather than shifting the median. That is precisely what the Reviewer does - the minimum A-condition of 0.30 becomes a minimum B-condition of 0.85.

7.2 Threshold Calibration Is Architecture-Dependent

The data indicates that the best threshold differs for Conditions B and C. In case of Condition B, thresholds getting higher generally mean improvement of scores up to the highest tested limit (0.85). For Condition C, the situation becomes the opposite after about 0.80: higher thresholds make the agent running the action worse by unnecessarily making it iterate longer on already long context traces.

Therefore, one single universal threshold is evidently not the right solution for systems with different architectures. A changing threshold considering the present context length or number of agent steps would enable Condition C to

get the Reviewer's error correction benefit on truly failing instances while not forcing iterations on instances that are already on the way to convergence.

7.3 Reviewer Contribution and Generator Quality

At first, the judge-separation outcome indicated a neat scaling scenario: when the generator was a powerful 32B model, the reviewer barely added anything ($\Delta = +0.003$); when the generator was a less powerful 7B model, the reviewer added a substantial $\Delta = +0.114$. The obvious fallout was that the reviewer's added value would be inversely proportional to generator quality, and, hence, the lightweight detection algorithm would probably be most beneficial when it runs on top of smaller, hardware-constrained models.

The cross-family research makes that story a bit more complicated. Of the four non-Qwen generators evaluated with the Qwen3-32B judge — DeepSeek-Coder 6.7B, CodeLlama 7B, Llama 3.1 8B and Mistral 7B — there is no monotonic relation between single-pass score and reviewer contribution. CodeLlama 7B has the weakest single-pass baseline in the set ($A = 0.355$) and exhibits a modest $\Delta = +0.076$. Mistral 7B has an intermediate baseline ($A = 0.511$) and shows the highest reviewer contribution of the whole research ($\Delta = +0.199$, $t = +3.85$, $p < 0.001$). DeepSeek-Coder 6.7B, which has a similar baseline to Mistral ($A = 0.529$), displays practically no reviewer effect ($\Delta = +0.021$, n.s.). Llama 3.1 8B, the strongest baseline of the four ($A = 0.568$), has $\Delta = +0.093$. The simple idea of "weakness produces lift" does not accommodate these data.

The contribution of the reviewer might rather depend on the generator's refinability or in other words, the degree to which a model's output is truly changed after being critiqued by the reviewer. There are two pieces of evidence which give this point of view some weight. First, DeepSeek-Coder is

incredibly indifferent to being refined: it invokes the refinement loop on 83% of the cases but each time produces almost the same result, so the reviewer's critique fails to bring about any measurable change in the scores. Second, Mistral 7B is the exact opposite: the 20:1 lift-to-regression ratio in the Qwen3-32B judge column points to the model that changes its response because of critique and barely, if at all, produces degraded output. Definability varies with family and is not something one can derive solely from the single pass baseline.

7.4 Bimodality in Condition C

The $C@0.85$ bimodal distribution is the most structurally compelling finding. It means the OpenHands agent is behaving on these 19 instances more like a salt-and-pepper distribution rather than a single smooth curve: either the agent solves the problem and thus yields a high-quality trace (14 instances) or it finds itself in a dead end without being able to produce a good trace even if the Reviewer forces many refine iterations (5 instances).

The failing instances are not randomly picked - rather, they are instances of more complex multi-file dependency chains where the agent gets trapped in loop paths understanding, collecting context without moving forward.

7.5 Limitations

The scores assigned by the reviewer are derived from a large language model. All the metrics of correctness mentioned in the paper owe to what the reviewer asserts rather than the patch's conformity to SWE-bench Lite's hidden tests. The paper evidence the reviewer refining its scoring, but the relation to genuine testing outcomes hasn't been quantified yet.

Condition C is still not yielding a definitive outcome on a genuine SWE-bench Lite test. Action agent executions have so far given rise to widely fluctuating scores. Some of them have performed well while others have failed. Although there is a hallucination bridge in the form of the architecture, there is still no documented instance of the reviewer identifying and rectifying a hallucinated action.

The 0.85 acceptance threshold was decided through qualitative observation and a rough two-point sensitivity check, and it was continuously used for all the subsequent rounds without being recalibrated. A separate calibration set should be prepared for the journal version of the paper, and the threshold should be fixed based on calibration data before any figures for the evaluation set are revealed.

The refine loop is not always generating beneficial results; in fact, between 4 and 33 per cent of cases per cell can be attributed to regressions. Moreover, the system lacks safeguards against the loop's transforming an already acceptable answer into a worsened one. A score-stagnation-based early stop rule would be the most straightforward next step in refinement.

CONCLUSIONS

This article presents a lightweight hallucination detection algorithm whose main part is Reviewer Agent embedded in a multi-agent code generation system guided by LangGraph. This detection method evaluates generated code in five thematic areas (Correctness, Edge Cases, Security, Code Quality, Relevance), opens a limited refinement loop based on the resulting confidence value, and does not require fine-tuning or internal logit access.

Additionally, the Hallucination Bridge is a separate extension that incorporates the very same logic to the stepwise action traces of an autonomous OpenHands coding agent.

Now the main claim is additionally backed by three nested rounds of evaluation. First, a round of 19 cases demonstrated that the refine loop serves the role of a quality floor at the calibrated acceptance threshold, the minimum accepted score getting raised from 0.30 to 0.85 and the standard deviation approximately halved. The paired replication on 50 cases revealed the first statistically significant result ($\Delta = +0.105$, $t = +2.27$, $p < 0.05$). The cross-family investigation using four non-Qwen architectures further verified that the Reviewer Agent's contribution is not limited to a single model family, and in fact, the Mistral 7B combination gave the lead single-cell result of this study ($\Delta = +0.199$, $t = +3.85$, $p < 0.001$).

The two results from the cross-family study change the way we should think about the work. The Reviewer's help does not go down in a single straight line as the generator gets weaker; it rather goes up or down with the amount of refinability, how much a model is open to responding to critique, a feature that is specific to a family and cannot be predicted from single-pass performance alone. The judge's influence can be broken down into a scale part, in which different judges give identical outputs systematically different absolute scores, and a lift part, in which different judges generate different paired deltas from the same generator output. Without distinguishing between these two parts, it is not possible to forecast how the algorithm will behave when a judge or a generator change happens at deployment.

Altogether, the pieces of evidence point to the main idea: a prompt-based, lightweight hallucination detection algorithm, when used as a Reviewer Agent in a multi-agent code generation system, can bring about measurable correctness gains on genuine software engineering issues without the need for fine-tuning, logit access, or an external verification setup.

LIST OF REFERENCES

1. Maynez J. et al. On Faithfulness and Factuality in Abstractive Summarization. ACL, 2020.
2. Ji Z. et al. Survey of Hallucination in Natural Language Generation. ACM Comput. Surv., 2023.
3. Lin S. et al. TruthfulQA: Measuring How Models Mimic Human Falsehoods. ACL, 2022.
4. Huang L. et al. A Survey on Hallucination in LLMs. arXiv:2311.05232, 2023.
5. Manakul P. et al. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection. EMNLP, 2023.
6. Ouyang L. et al. Training LMs to Follow Instructions with Human Feedback. NeurIPS, 2022.
7. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP. NeurIPS, 2020.
8. Chen M. et al. Evaluating Large Language Models Trained on Code (Codex). arXiv:2107.03374, 2021.
9. Jimenez C.E. et al. SWE-bench: Can LLMs Resolve Real-World GitHub Issues? ICLR, 2024.
10. Wu Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
11. Hong S. et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR, 2024.

12. Qian C. et al. ChatDev: Communicative Agents for Software Development. ACL, 2024.
13. Yao S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
14. Wang X. et al. OpenHands: An Open Platform for AI Software Developers. arXiv:2407.16741, 2024.
15. Madaan A. et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023.
16. Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
17. Pan L. et al. Automatically Correcting Large Language Models. TACL, 2024.
18. Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023.
19. Yang A. et al. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.